编译原理
6. Activation Record

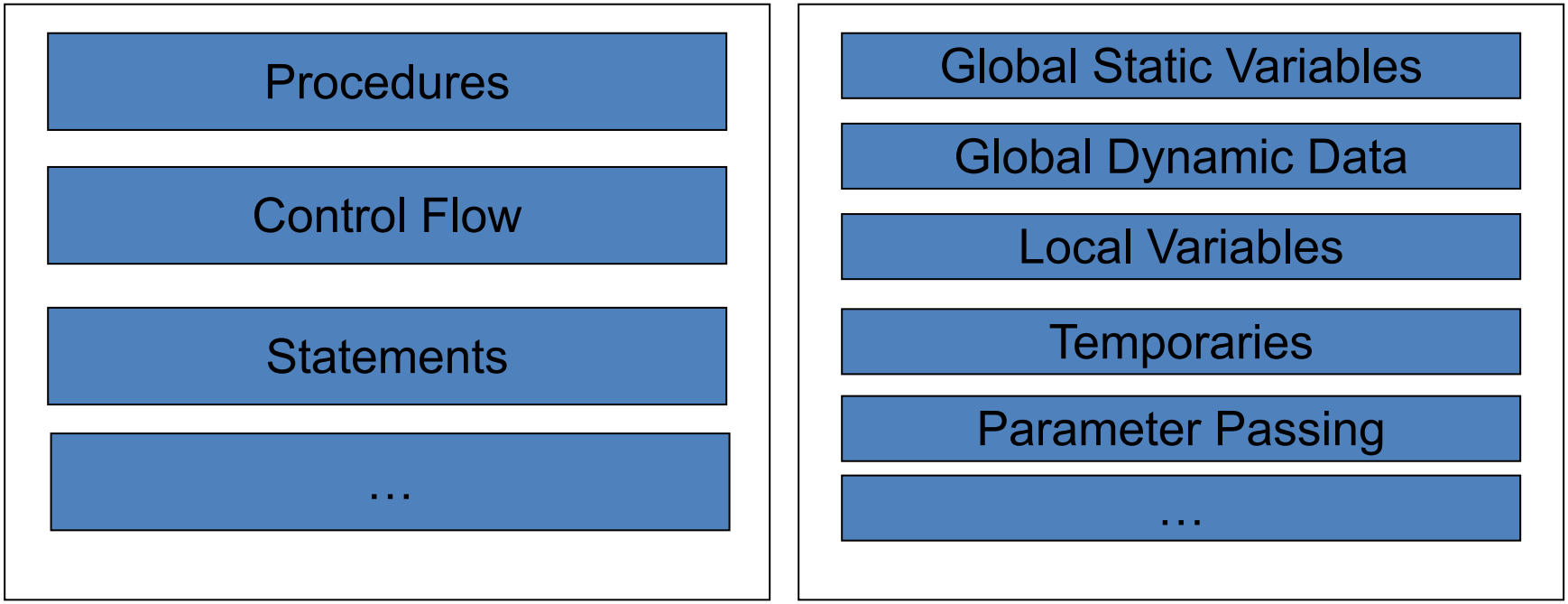
rainoftime.github.io
浙江大学
计算机科学与技术学院

课程内容

1. Introduction
2. Lexical Analysis
3. Parsing
4. Abstract Syntax
5. Semantic Analysis
6. **Activation Record**
7. Translating into Intermediate Code
8. Basic Blocks and Traces
9. Instruction Selection
10. Liveness Analysis
11. Register Allocation
13. Garbage Collection
14. Object-oriented Languages
18. Loop Optimizations

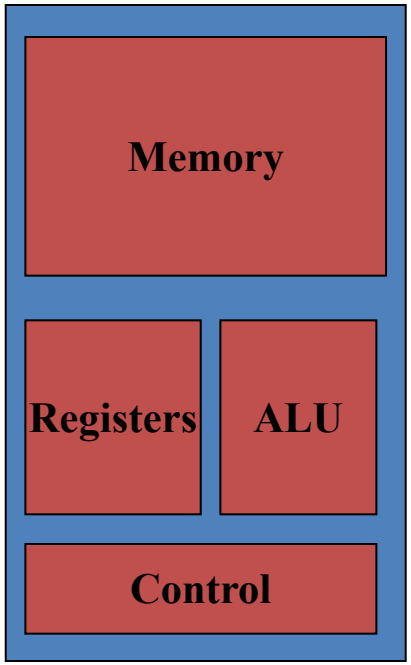
Run-time Environments

- A compiler should translate all "CODE" to assembly instructions and allocate space for "DATA"



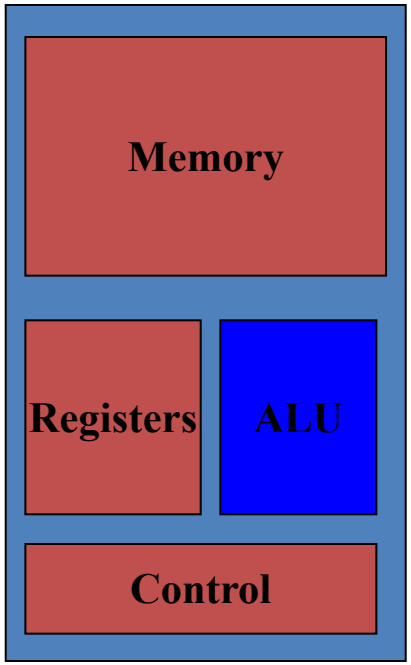
Overview of a Modern Processor

- ALU 算术逻辑单元
- Control
- Memory
- Registers



Modern Processor: Arithmetic and Logic Unit

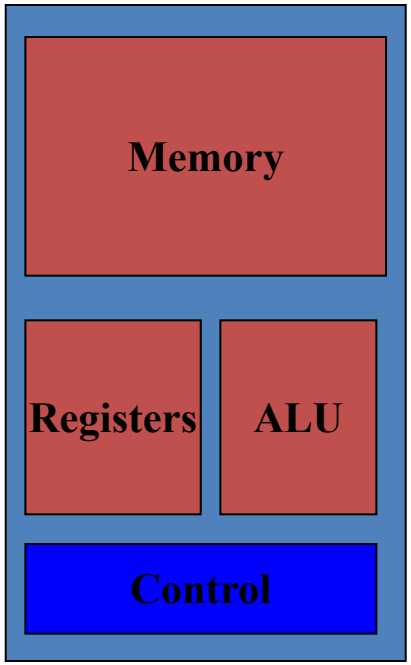
- **Most arithmetic and logic operation**
  - add rax, rbx ; rax = rax + rbx
  - mov rax, rbx ; rax = rbx
- **Operands:**
  - immediate 立即数
  - register
  - memory



Modern Processor: Control

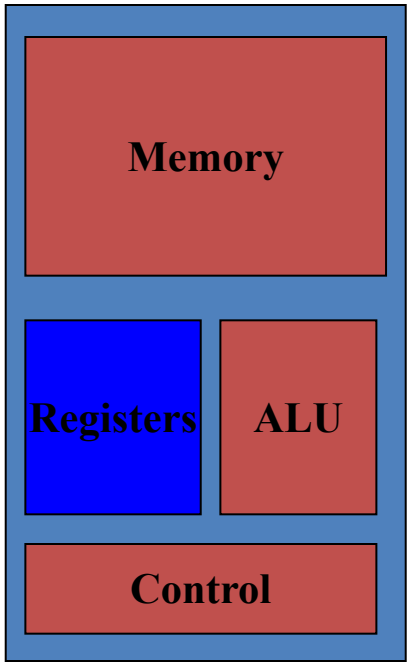
- **Executing instructions**
  - Instructions are in memory (pointed by PC)

```
for (;;)
  instruction = *PC;
  PC++;
  execute (instruction);
```



Modern Processor: Registers

- **Limited but high-speed**
  - More on RISC than x86
- **Most are general-purpose**
  - But some are of special use
  - E.g., rsp, rbp in x86-64

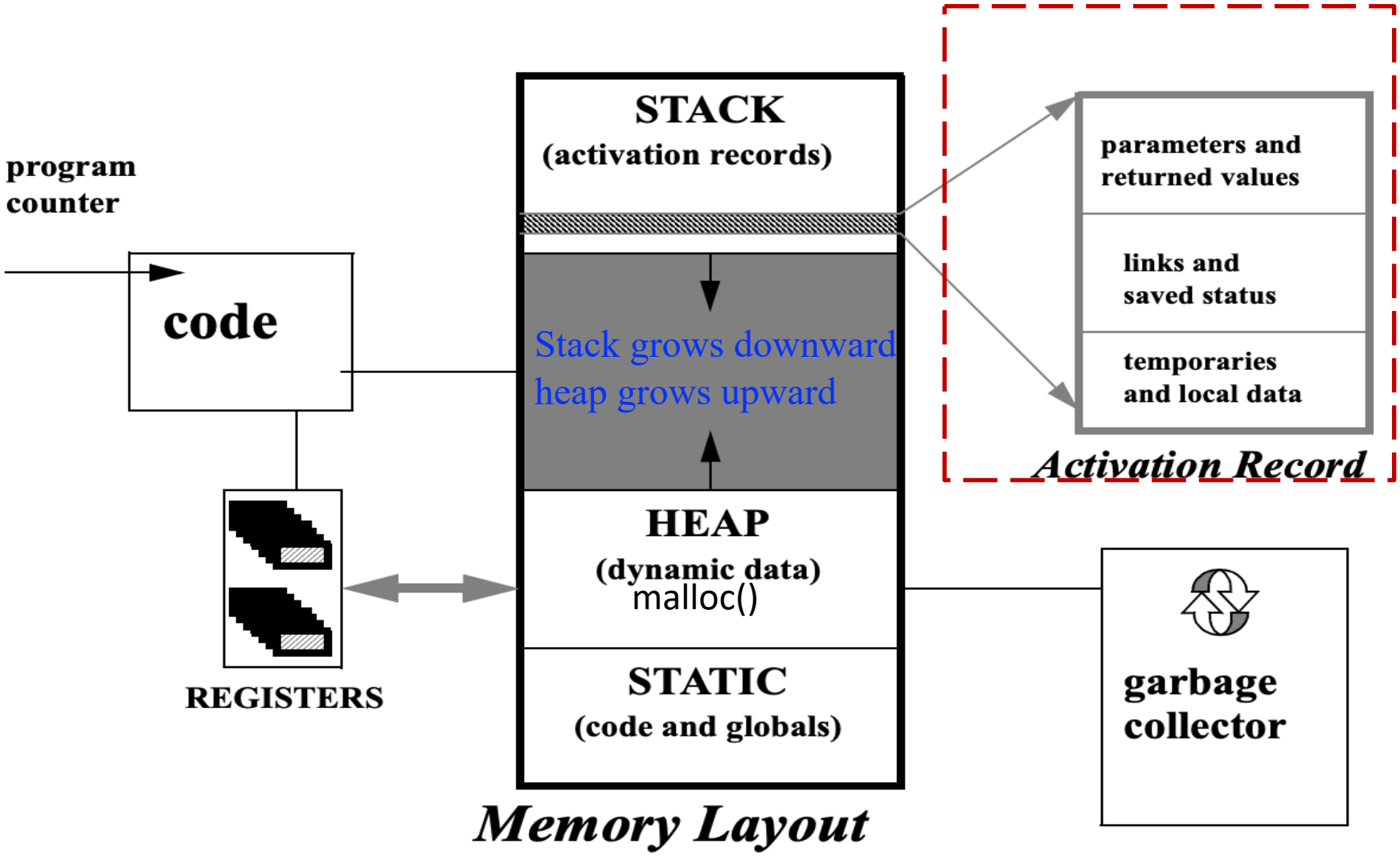


Modern Processor: Memory

- **Address space is the way how programs use memory**

Dest, Src	C Analog
`mov rax, 0x4`	`temp = 0x4;`
`mov [rax], -147`	`*p = -147;`
`mov rax, [rdx]`	`temp = *p;`

Runtime Memory Layout of Programs



Focus of this Lecture: Activation Record

When We Talk about Function Calls

- **Application Programming Interface**
  - Interfaces between source programs

- **Application Binary Interface (e.g., x86 ABI)**
  - Contracts between binary programs
    - Even compiled from other languages
  - Conventions on low-level details
    - How to pass arguments?
    - how to return values?
    - how to make use of registers?
    - …

Outline

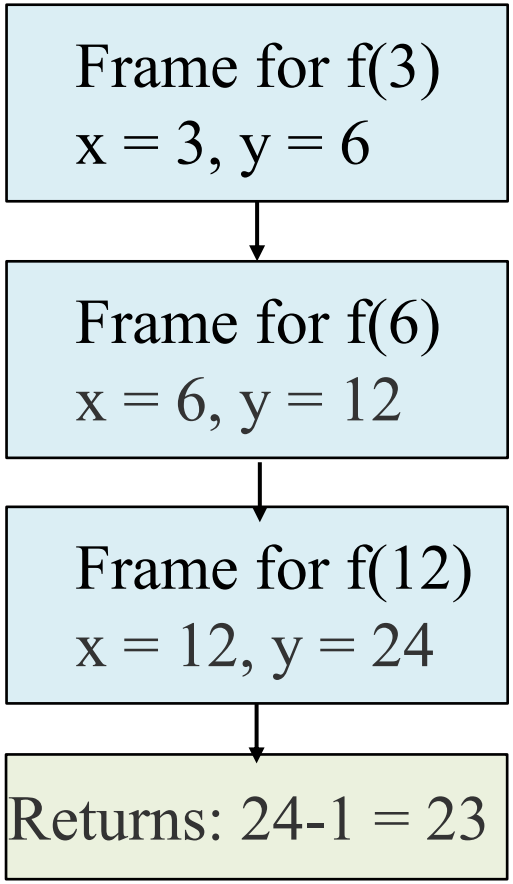
1. Stack Frame
2. Use of Registers
3. Frame-Resident Variables
4. Block Structure
5. Block Structure: Imple.
6. A Typical Layout for Tiger

1. Stack Frame

Activation Record/Stack Frame

- **An invocation of function P is an activation of P**

```
function f(x:int): int =
  let var y :== x+x
   in if y<10
     then f(y)
     else return y-1
  end
```

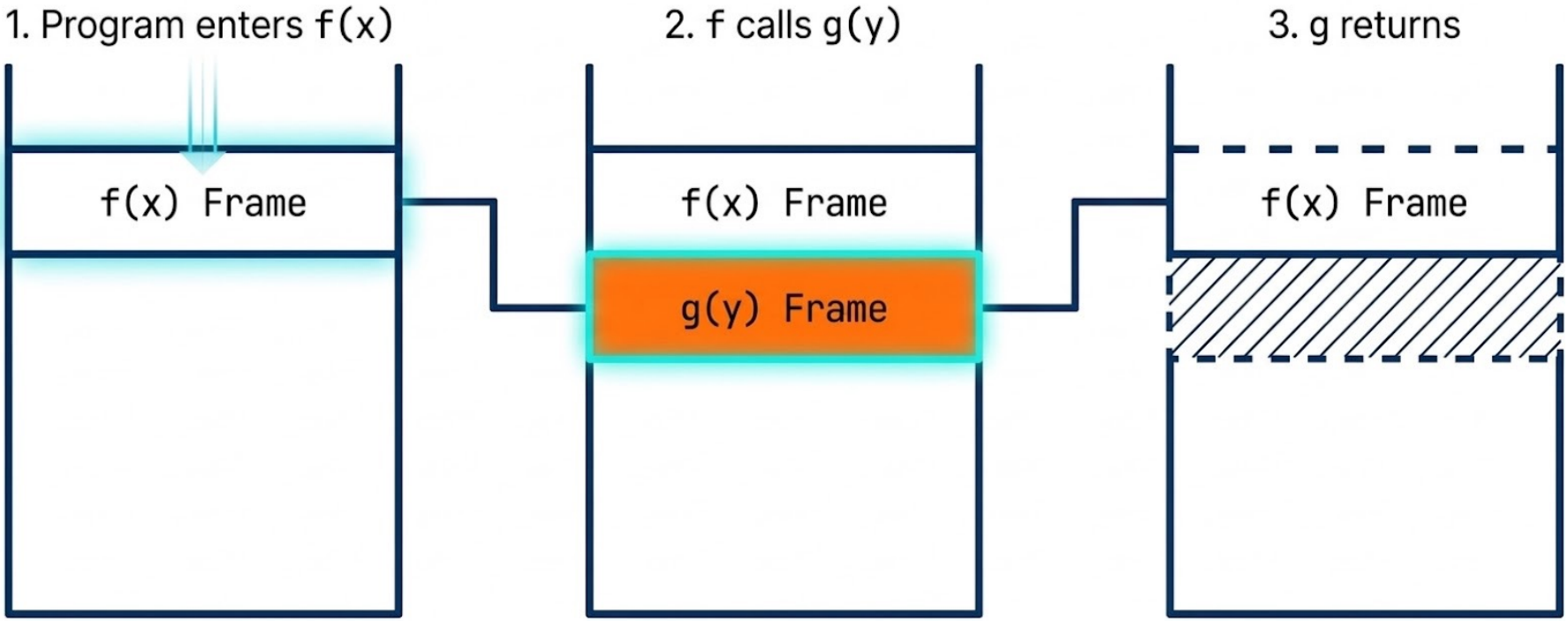


- Each call to f creates a new instance of parameter x
- Each entry to the body creates a new local variable y

Call trace:
f(3) → f(6) → f(12) → return 23

Example: FIFO Principle of Function Calls

- Local variables are created on function entry and destroyed on exist



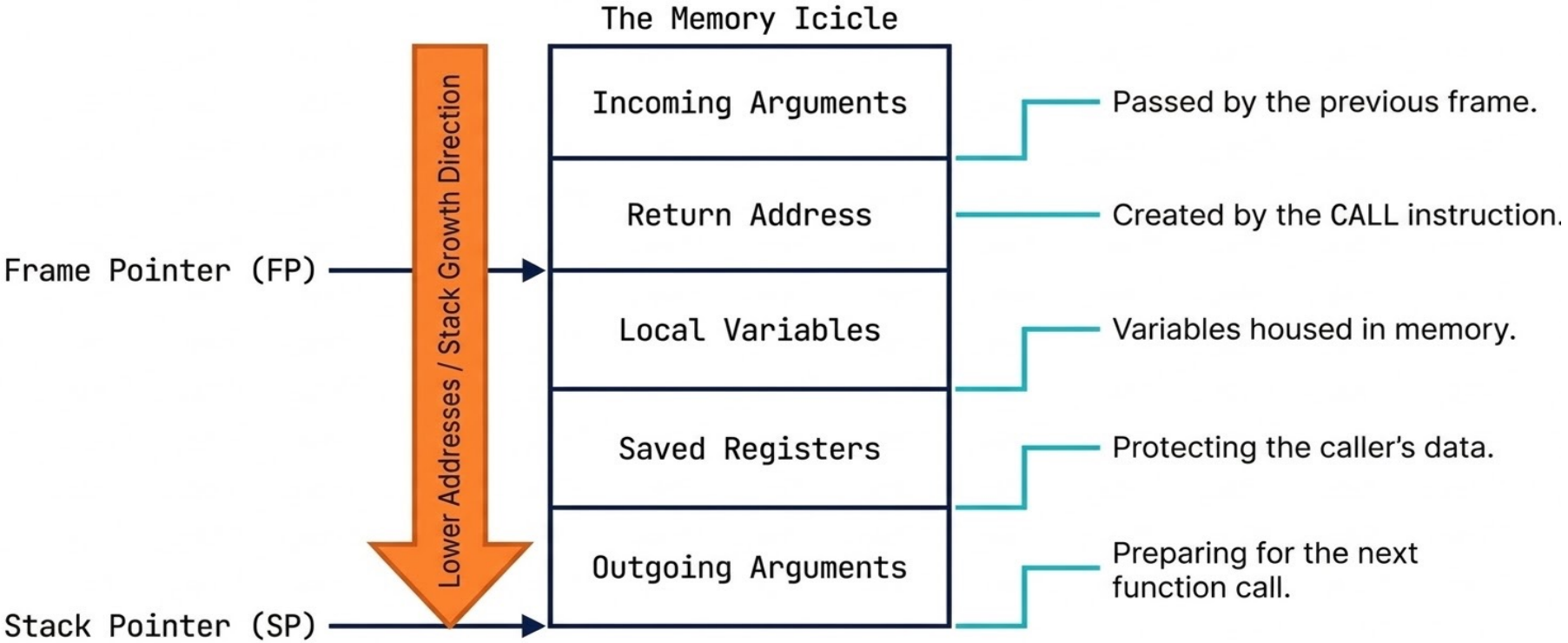
Activation Record/Stack Frame

- Activation record or stack frame: a piece of memory on the stack for a function
- The stack frame connects the caller to the callee, e.g.,
  1. Relevant machine state (saved registers, etc.)
  2. Space for return value
  3. Space for local data
  4. Pointer to activation for accessing **non-local** data

How to **layout** the activation record so that the caller and callee can **communicate properly** and **efficiently**?

Example: The Contents of Typical ARs

- The run-time stack grows toward smaller adddress



What about the Globals

- All references to a global variable point to the same object
  - Can't store a global in an activation record
- Globals are assigned a fixed address once
  - Variables with fixed address are "statically allocated"
- Depending on the language, there may be other statically allocated values
  - In Tiger, string constants are allocated as globals

What about the Heap Storage

- A value that outlives the procedure that creates it cannot be kept in the AR

  method foo() { new Bar }

  The Bar value must survive deallocation of foo's AR

- Languages with dynamically allocated data use a heap to store dynamic data
  - Records and arrays contents in Tiger have infinite extent (They are allocated in heap)

Frame Pointer and Stack Pointer

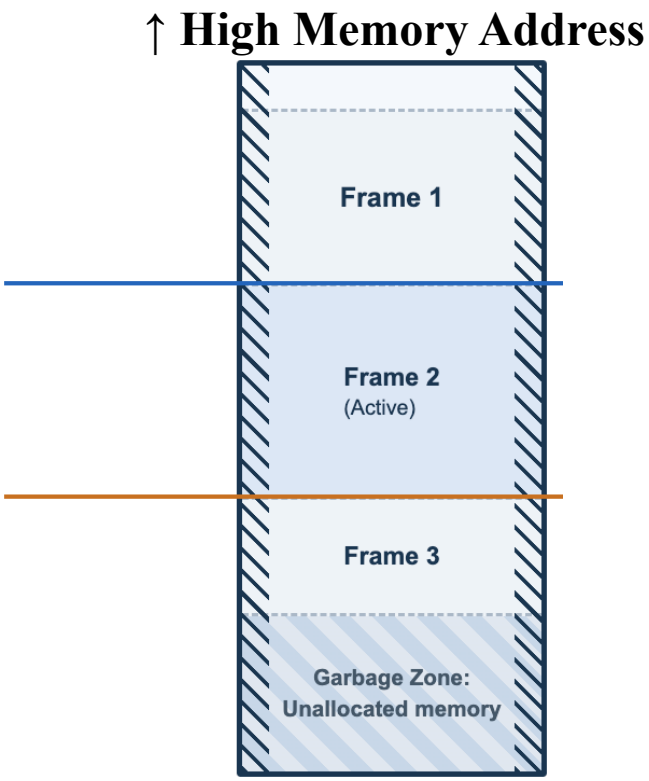
- Frame pointer (FP) 帧指针, “基址寄存器(base pointer)” 指向当前函数栈帧的基址
- Stack pointer (SP), 栈指针. 指向栈顶

Frame Pointer (FP)

- Anchors the current frame.
- Fixed at SP + framesize.

Stack Pointer (SP)

- Points to the lowest allocated location.
- Moves **downward** by the exact **frame size** upon function entry.



Example: FP and SP

每个函数的机器码都遵循 **Prologue → Body → Epilogue** 结构

```c
int g(long long x)
{
  return x + 1;
}

void f() {
  g(10086);
}
```

```asm
g(long long): # @g(long long)
    push rbp          ; 保存旧 FP
    mov rbp, rsp      ; FP = SP（建立帧基址）
    mov qword ptr [rbp - 8], rdi
    mov rax, qword ptr [rbp - 8]
    add rax, 1
    pop rbp           ; 恢复旧 FP
    Ret               ; 返回 caller

f(): # @f()
    push rbp          ; 保存旧 FP
    mov rbp, rsp      ; FP = SP（建立帧基址）
    mov rdi, 10086
    call g(long long)
    pop rbp           ; 恢复旧 FP
    ret
```

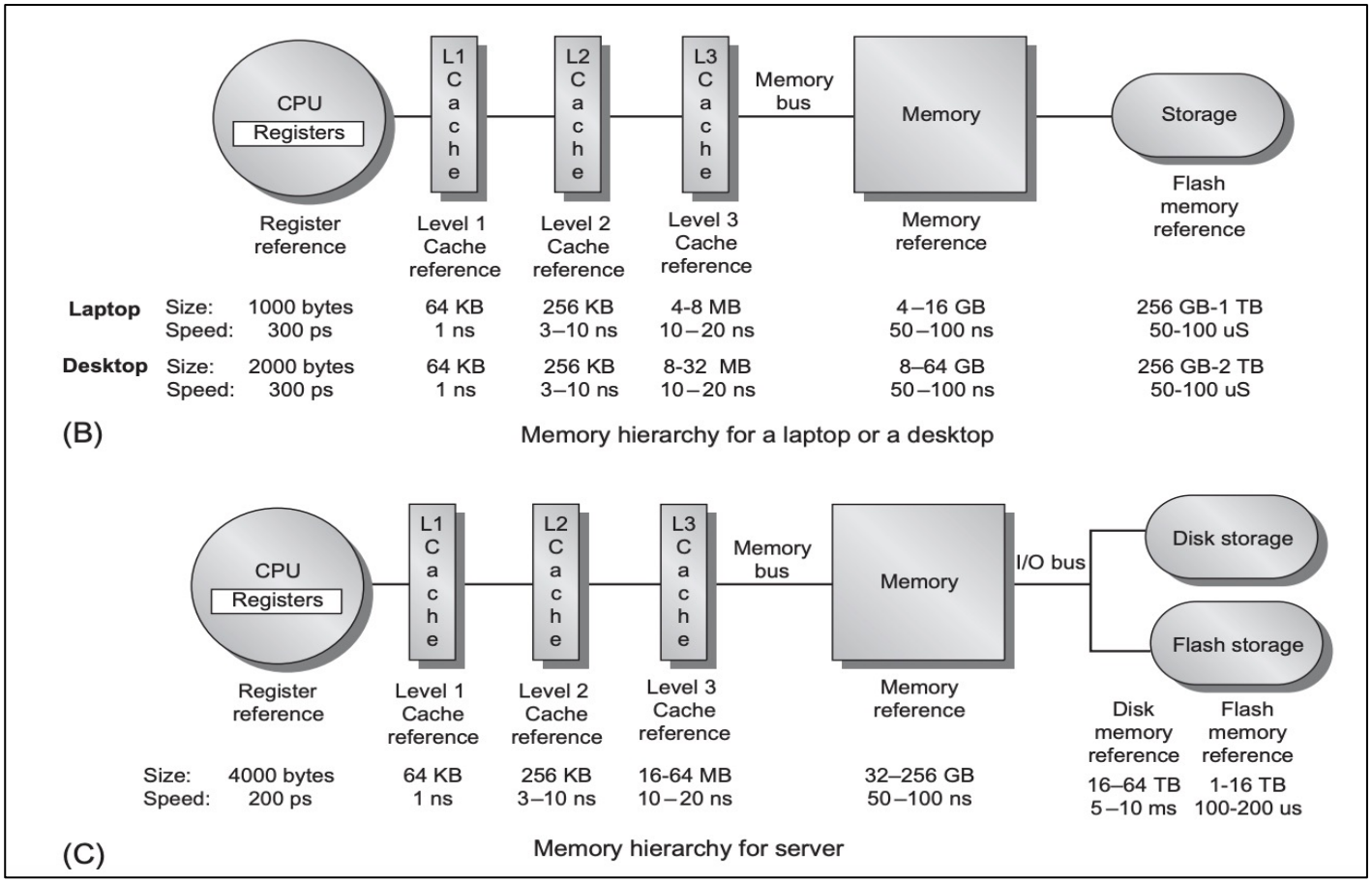
实际应用：许多现代编译器在优化模式下**省略 FP**（-fomit-frame-pointer），因为编译器能静态计算所有偏移。但调试信息（DWARF）仍需记录帧布局。

2. Use of Registers

How to reduce memory traffic?

Recap: Memory Hierarchy

- Accessing registers is MUCH faster than accessing memory!



Computer Architecture Quantitative Approach, Sixth Edition

Reducing Memory Allocation in Stack Frame

- **Problem**: Putting everything in the stack frame can cause the memory traffic
- **Solution**: Hold as much of the frame as possible in registers
  - **(Some) function parameters**
  - **Function return address**
  - **Function return value**
  - **(Some) Local variables**
  - **(Some) Intermediate results of expressions (temporaries)**

Using Registers: 1. Parameter Passing

- **Tiger的参数方式: Call-by-value**
  - Values of the actual arguments are passed and established as values of formal parameters.
  - Modification to formals have no effect on actuals

```
function swap(x : int, y : int) =
    let var t : int := x in x := y; y := t end
```

- Parameters are passed on the stack for most machines designed in 1970s
  - **Problem**: caused the memory traffic

Using Registers: 1. Parameter Passing

- **Problem**: passing parameter stack causes memory traffic
- **Solution**: parameter-passing convention on modern machines
  - The first k arguments (k = 4 or 6) are passed in registers
    - **X86-64:** rdi, rsi, rcx, rdx; **ARM：** r0~r3
  - The rest of the arguments: passed on the stack

```
int g(long long x) {
  return x + 1;
}

void f() {
  g(10086);
}
```

```
g(long long): # @g(long long)
      push rbp
      mov rbp, rsp
      mov qword ptr [rbp - 8], rdi
      mov rax, qword ptr [rbp - 8]
      add rax, 1
      pop rbp
      ret
f(): # @f()
      push rbp
      mov rbp, rsp
      mov rdi, 10086
      call g(long long)
      pop rbp
      ret
```

rdi传第一个参数

Using Registers: 1. Parameter Passing

- The need for **"saving" the status** of registers

```
int calculate(int a) {
        int x = a * 2;
        int z = helper(x);
        return a * z;
}
```

- 假设calculate使用寄存器 ***r*** 自保存参数**a**的值。在调用函数 ***helper***返回后，需要用到**a**的值。
- 如果***helper***函数会修改寄存器***r***的值，如何确保在 helper 函数调用后，**a** 的值是原来的？

Using Registers: 1. Parameter Passing

- The need for **"saving" the status** of registers

- **r** is **caller-save** if the caller must save and restore it
  - E.g., rdi, rsi, rcx, rdx

  调用函数负责保存,它在函数调用后 还需使用的寄存器的值

- **r** is **callee-save** if it is the responsibility of the callee
  - E.g., rbp (frame pointer)

  被调用函数负责保存和恢复 它要使用的寄存器的原始值

Example: Caller-Save vs. Callee-Save

```
int calculate(int a, int b) {
        int x = a * 2;
        int y = b + 10;
        int z = helper(x);
        return y * z;
}
```

1. 将 y 存储在caller-save寄存器 rsi 中。
2. 在调用 helper 前，我们显式地将 rsi 的值保存到栈上（ push 指令）

3. 调用返回后，我们从栈上恢复 rsi 的值（ pop 指令）
4. 这样可以确保即使 helper 函数修改了 rsi 的值，我们也能恢复原来的 y 值

```
Caller-Save (Using RSI)
calculate:
    # a in rdi, b in rsi
    imul rax, rdi, 2 # x = a * 2
    add rsi, 10      # y = b + 10

    push rsi         # Save y (caller-save)

    mov rdi, rax     # x as argument
    call helper

    pop  rsi         # Restore y !
    imul rax, rsi    # y * z

ret
```

Example: Caller-Save vs. Callee-Save

```
int calculate(int a, int b) {
        int x = a * 2;
        int y = b + 10;
        int z = helper(x);
        return y * z;
}
```

1. 将 y 存储在callee-save寄存器 r12 中
2. 调用 helper 函数时不需要额外保存 r12 的值。

3. helper 函数有责任保存和恢复它使用的所有callee-save寄存器，包括 r12。
4. 因此，即使 helper 内部修改了 r12，在返回时也会恢复原值。

Callee-Save (Using R12)

```
calculate:
    # a in rdi, b in rsi
    imul rax, rdi, 2          # x = a * 2
    lea r12, [rsi + 10]       # y = b + 10

    # No need to save r12 (callee-save)
    mov rdi, rax              # x as argument
    call helper
    imul rax, r12             # y * z
    ret

helper:
    push r12                  # Save r12

    # Function logic...

    pop r12                   # Restore r12
 ret
```

Caller-Save vs. Callee-Save (供参考)

特性	调用者保存 (Caller-Save)	被调用者保存 (Callee-Save)
保存责任	调用函数负责保存它在调用后还需要的寄存器值	被调用函数负责保存它要修改的被调用者寄存器值
保存位置	每个函数调用点前后	被调用函数的开始和结束处
x86_64寄存器	RAX, RCX, RDX, RSI, RDI, R8-R11	RBX, RBP, RSP, R12-R15
优点	只保存实际需要的寄存器 叶子函数不需要保存操作 适合短期临时变量	多次调用只需保存一次 适合长期存活的值 减少调用点的代码重复
缺点	每个调用点都需要保存/恢复代码, 导致代码冗余. 频繁调用时效率低	必须保存所有使用的被调用者寄存器, 即使调用者不需要也要保存. 简单函数有额外开销
适用场景	•临时计算结果 •短期变量 •函数调用不频繁的代码	•跨多个函数调用的变量 •频繁调用其他函数的代码 •长期存活的局部变量

了解这两种约定对于理解汇编代码、优化性能和调试程序都非常重要。

Using Registers: 1. Parameter Passing

- The need for "saving" the status of registers
- **New problem:** extra memory traffic caused by passing arguments in registers!

- **r** is **caller-save** if the caller must save and restore it
  - E.g., rdi, rsi, rcx, rdx
- **r** is **callee-save** if it is the responsibility of the callee
  - E.g., rbx, rbp (frame pointer)

如果 *f* 在用rdi传参前，要先把rdi当前的值存入stack frame (调用完再恢复)，那最终并没有避免stack frame的访存操作!

Example: Problem of Parameter Passing

```
1:  f(int a) {
2:      int z = ...
3:      h(z);
4:      ...
5:      int t = a + 2;
6:      …
}
```

- Suppose ***f*** received its parameter in register $\mathbf{r_1}$
- Suppose ***f*** passes ***z*** to the callee ***h*** via register $\mathbf{r_1}$
- ***f*** should save the old contents of $\mathbf{r_1}$ in stack frame before calling ***h*** **!**

Extra memory traffic caused by passing arguments in registers?!

Such memory traffic was supposedly avoided
by passing arguments in registers!

Using Registers: 1. Parameter Passing (cont'd)

```
1:  f(int a) {
2:      int z = ...
3:      h(z);
4:      ...
5:      int t = a + 2;
6:      …
}
```

- Suppose ***f*** received its parameter in register $\mathbf{r_1}$
- Suppose ***f*** passes ***z*** to the callee ***h*** via register $\mathbf{r_1}$
- ***f*** should save the old contents of $\mathbf{r_1}$ in stack frame before calling ***h*** **!**

🤔How to avoid the extra memory traffic?

Using Registers: 1. Parameter Passing (cont'd)

```
1:  f(int a) {
2:      int z = ...
3:      h(z);
4:      ...
5:      int t = a + 2;
6:      …
}
```

- Suppose ***f*** received its parameter in register $\mathbf{r_1}$
- Suppose ***f*** passes ***z*** to the callee ***h*** via register $\mathbf{r_1}$
- ***f*** should save the old contents of $\mathbf{r_1}$ in stack frame before calling ***h*** !

How to avoid the extra memory traffic?

1. **If the parameter *a* is a dead variable at the point where *h* is called, then *f* can overwrite $\mathbf{r_1}$ without saving it.**

例如: 在调用h后a的值不再使用(不存在第5行)

Using Registers: 1. Parameter Passing (cont'd)

```
1:  f(int a) {
2:      int z = ...
3:      h(z);
4:      ...
5:      int t = a + 2;
6:      …
}
```

- Suppose ***f*** received its parameter in register $\mathbf{r_1}$
- Suppose ***f*** passes ***z*** to the callee ***h*** via register $\mathbf{r_1}$
- ***f*** should save the old contents of $\mathbf{r_1}$ in stack frame before calling ***h*** !

How to avoid the extra memory traffic?

2. Use global register allocation: different functions use different set of registers to pass arguments

例如: f可用寄存器$\mathbf{r_1}$接收参数，但通过寄存器$\mathbf{r_2}$给h传参

Using Registers: 1. Parameter Passing (cont'd)

```
1:  f(int a) {
2:      int z = ...
3:      h(z);
4:      ...
5:      int t = a + 2;
6:      …
}
```

- Suppose ***f*** received its parameter in register $\mathbf{r_1}$
- Suppose ***f*** passes ***z*** to the callee ***h*** via register $\mathbf{r_1}$
- ***f*** should save the old contents of $\mathbf{r_1}$ in stack frame before calling ***h*** !

How to avoid the extra memory traffic?

3. Leaf procedures:不调用其他过程的为叶子过程(Leaf procedure)。叶子过程不必将传入的参数保存到存储器中

Using Registers: 1. Parameter Passing (cont'd)

How to avoid the extra memory traffic?

1. Parameter *x* is a dead variable at the point where *h(z)* is called. Then *f(x)* can overwrite $\mathbf{r_1}$ without saving it.

2. Use **global register allocation:** Different functions use different set of registers to pass arguments

3. Leaf procedures: parameters of leaf procedures can be allocated in registers without causing any extra memory traffic

4. Use **register windows** (as on SPARC): Each function invocation can allocate a fresh set of registers

Using Registers: 2. Return Address

- **Return address**

  When the function g calls the function f, f needs to know where to return.

- **Two approaches**
  - **Old approach (1970s):** Push return address on stack via call instruction
  - **Modern approach:** Put return address in a designated register (e.g., rip 指令指针寄存器)

Using Registers: 3. Return Value

- **Return value**

  Placed in designated register by callee function.

  – X86-64系统整型返回值： rax

```
int add(int a, int b) {
   // 返回值将放入rax寄存器
   return a + b;
}
int main() {
   // 调用add后从rax获取返回值
   int result = add(5, 3);
   return 0;
}
```

不同数据类型的返回处理

- 对于整数和指针类型，通常使用rax寄存器
- 对于浮点数返回值，通常使用XMM寄存器（如XMM0）
- 对于较大的数据结构，可能通过栈返回，而不是寄存器

Using Registers: 4. Locals and Temporaries

- **(Some) Local variables**
- **(Some) Intermediate results of expressions (temporaries)**

```
int calculate(int a, int b) {
    int x = a + 5;    // 局部变量x可能被分配到寄存器
    int y = b * 2;    // 局部变量y可能被分配到另一个寄存器
    return x + y;     // 直接使用寄存器中的值进行计算
}
```

To be discussed in register allocation section!

3. Frame-Resident Variables

既然很多地方都可以用寄存器，那还需要在stack frame中分配内存空间吗?

Frame-Resident Variables

A variable will be allocated in stack frames because

1. It is *passed by reference*, so it must have a memory address
2. Its *address is taken*, e.g., &a in the C language
3. It is accessed by a procedure nested inside the current one;
4. The value is too big to fit into a single register
5. The variable is an array, for which address arithmetic is necessary to extract components
6. The register holding the variable is needed for a specific purpose, such as parameter passing (as described above)
7. The are too many locals and temporaries – "spill" [To be discussed in Register Allocation]

Frame-Resident Variables

A variable will be allocated in stack frames because

1. It is ***passed by reference***, so it must have a memory address

```
void increment(int *ptr) {
   (*ptr)++;              // 修改指针指向的值
}

void main() {
   int x = 10;            // x 需要在栈中有位置，以便获取其地址
   increment(&x);         // 传递 x 的地址
   …                      // 现在 x 的值为 11
}
```

Frame-Resident Variables

A variable will be allocated in stack frames because

1. It is ***passed by reference***, so it must have a memory address
2. Its ***address is taken***, e.g., &a in the C language

```
void process() {
   int count = 0;          // 需要在栈中分配，因为下面会取其地址
   int *ptr = &count;      // 取 count 的地址
   *ptr = 42;              // 通过指针修改值
}
```

Dest, Src	C Analog
`mov rax, 0x4`	`temp = 0x4;`
`mov [rax], -147`	`*p = -147;`
`mov rax, [rdx]`	`temp = *p;`

Frame-Resident Variables

A variable will be allocated in stack frames because

1. It is ***passed by reference***, so it must have a memory address
2. Its ***address is taken***, e.g., &a in the C language
3. It is **accessed by a procedure nested inside the current one**

```
void outer() {
   int shared = 10;    // 在栈中分配
   void (*inner)(void) = function() {
      printf("%d", shared); // 访问外部函数的变量
   };
   inner(); // 调用内部函数
}
```

Frame-Resident Variables

A variable will be allocated in stack frames because

1. It is *passed by reference*, so it must have a memory address
2. Its *address is taken*, e.g., &a in the C language
3. It is accessed by a procedure nested inside the current one
4. The value is **too big to fit into a single register**

```
struct LargeStruct {
    double values[10];
    char name[100];
};
void process() {
  // 太大，无法放入寄存器
    struct LargeStruct data;
    data.values[0] = 3.14;
}
```

Frame-Resident Variables

A variable will be allocated in stack frames because

1. It is *passed by reference*, so it must have a memory address
2. Its *address is taken*, e.g., &a in the C language;
3. It is accessed by a procedure nested inside the current one;
4. The value is too big to fit into a single register;
5. The variable is **an array**, for which address arithmetic is necessary to extract components;

```
void calculateSum() {
    int numbers[100];  // 栈中分配
    for (int i = 0; i < 100; i++)
        numbers[i] = i; // 访问数组需地址计算: base + i*sizeof(int)
```

Frame-Resident Variables

A variable will be allocated in stack frames because

1. It is *passed by reference*, so it must have a memory address
2. Its *address is taken*, e.g., &a in the C language
3. It is accessed by a procedure nested inside the current one;
4. The value is too big to fit into a single register
5. The variable is an array, for which address arithmetic is necessary to extract components
6. The register holding the variable is needed for a specific purpose, such as parameter passing (as described above)
7. The are **too many locals and temporaries – "spill"**

   [To be discussed in Register Allocation]

Escape Variables

- **The variable escapes for any of the reasons**
  1. It is *passed by reference*
  2. Its *address is taken*, e.g., &a in C;
  3. It is accessed from a nested function

```
void f() {
   int x = 10;
    int *p = &x; // x 逃逸
}
```

```
void increment(int &n) { n++; }
void g() {
   int y = 5;
   increment(y); // y 逃逸
 }
```

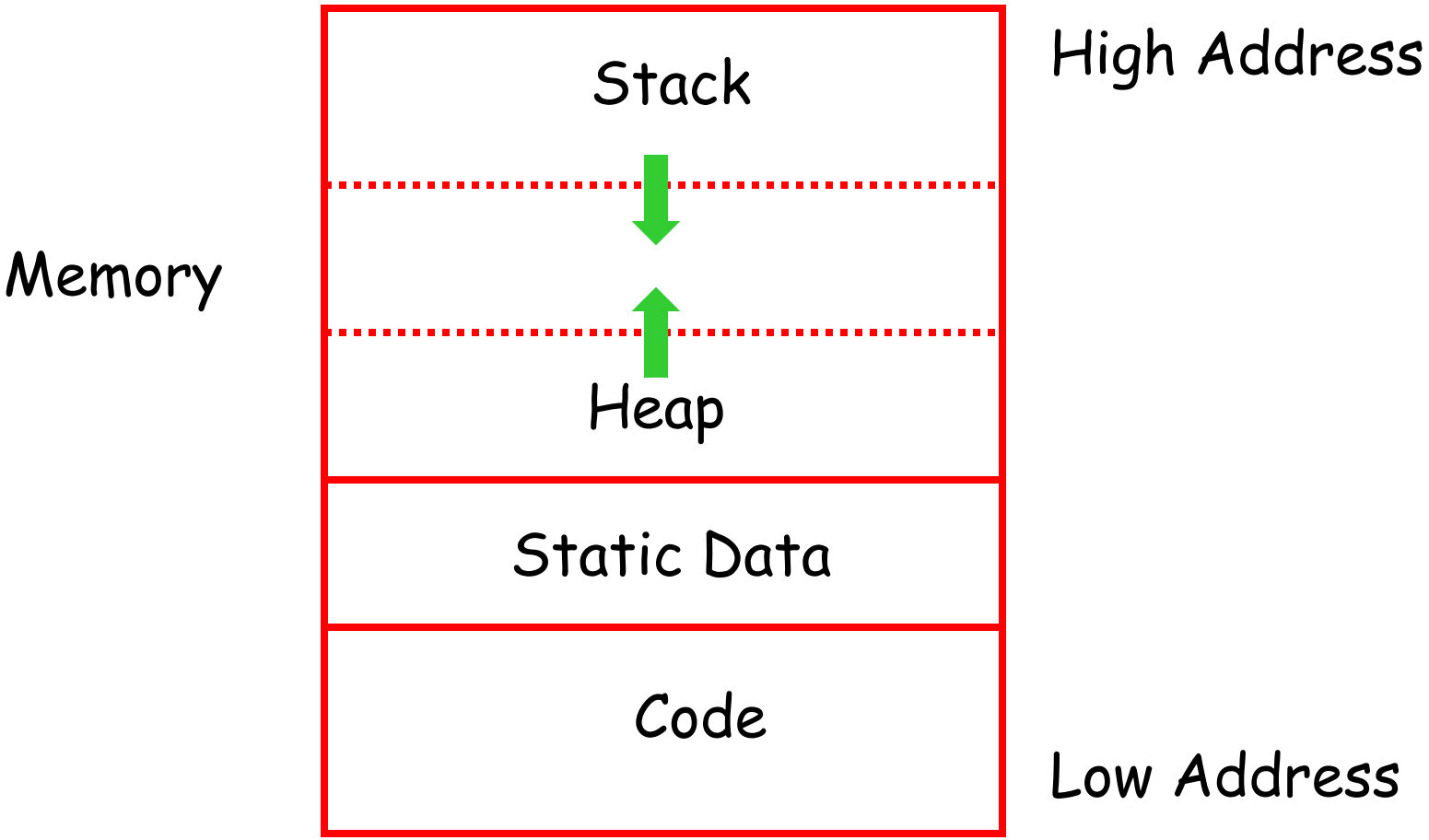
```
function outer() =
   let var z := 42
      function inner() =
         z := z + 1 // z 逃逸
     in
         inner();
         z
end
```

Summary

- **Registers hold**
  - Some parameters, Return address, Return value
  - Some local variables and temporaries
- **Stack frame holds**
  - Variables passed by reference or have their address taken (&)
  - Variables that are accessed by procedures nested within current one.
  - Variables that are too large to fit into register file.
  - Array variables (address arithmetic needed to access array elements).
  - Spilled registers (Too many local variables to fit into register file, so some must be stored in stack frame)

Thank you all for your attention

Memory Layout with Heap



Notes

- The code area contains object code
  - For most languages, fixed size and read only
- The static area contains data (not code) with fixed addresses (e.g., global data)
  - Fixed size, may be readable or writable
- The stack contains an AR for each currently active procedure
  - Each AR usually fixed size, contains locals
- Heap contains all other data
  - In C, heap is managed by *malloc* and *free*